Cohort 17 – Smart Streetlight Fault Detection

SDG Goal: SDG 9 – Industry, Innovation and Infrastructure

Cohort Number: 17

Submission Date: 13rd March of 2026

Team Members and Roles

USN	Name	Role
2AV22CS005 1HK22AI063 1HK22EC051	Amruta Suresh Aralelimath Hafeeza Tasneem Hajira Shahanz A	EDA + Data preprocessing
1ST22CS240 1ST22CS241 1MV22EC056 4VM22IS004	Vasanth Kumar H R Vibhashree R Harshith G K Benaka M	ML Engineer
1HK22AI062 1GC22CS096 1EW22IS070 4AD22EC062	Zeba Shine Safdar Iqbal Nakshatra S Nayana M B	MTTD+ Polling Frequency Analysis
1DT22IC003 1GC23CS419 1GC22CS068	Akshay S Tasaduque Ullah Mohammed Umar Maniyar	Documentation
1RI22CS108	Sridhar Y Gowdar	Git Hub Repo

Cohort Leaders

USN	Name	Role
1ST22CS240 2AV22CS005 4VM22IS004 1RI22CS108	Vasanth Kumar H R Amruta Suresh Aralelimath Benaka M Sridhar Y Gowdar	President Vice President Secretary Deputy Secretary

ABSTRACT

This project focuses on detecting faults in smart streetlight systems using machine learning techniques applied to Internet of Things (IoT) sensor data. Urban lighting infrastructures increasingly use IoT-enabled devices to monitor operational parameters such as voltage, current, power consumption, temperature, and light intensity. These sensors continuously generate data that can be analyzed to determine whether a streetlight is functioning normally or experiencing faults.

The collected sensor data is compiled into a structured dataset and undergoes preprocessing and cleaning to ensure quality and consistency. Missing values are handled using appropriate imputation techniques, and the dataset is normalized to improve model performance. Exploratory Data Analysis (EDA) is performed to understand the distribution of sensor readings and identify patterns associated with faulty streetlight behavior.

Machine learning classification models are then trained to categorize streetlight states as either operational or faulty. The system aims to detect failures quickly and efficiently, reducing maintenance delays and improving urban infrastructure reliability. The model performance is evaluated using Mean Time to Detect (MTTD), which measures how quickly the system can identify faults after they occur. This project contributes to improving smart city infrastructure and supports Sustainable Development Goal 9: Industry, Innovation, and Infrastructure.

1. INTRODUCTION

Street lighting plays a critical role in ensuring public safety, traffic visibility, and urban infrastructure efficiency. In modern smart cities, streetlights are increasingly integrated with IoT sensors and communication systems that enable remote monitoring and management. These smart streetlights collect operational data such as electrical parameters, environmental conditions, and device status information.

However, faults in streetlight systems—such as bulb failures, power supply issues, or sensor malfunctions—can lead to reduced lighting efficiency, increased maintenance costs, and potential safety risks. Traditional fault detection methods often rely on manual inspection or delayed reporting, which can result in prolonged downtime.

Machine learning provides an intelligent solution by analyzing IoT sensor data to automatically detect anomalies and predict faulty conditions. By training models on historical sensor data, it is possible to identify patterns associated with malfunctioning streetlights and trigger early maintenance alerts.

This project applies a data-driven approach to develop a machine learning model capable of classifying streetlight status based on IoT sensor data. Such systems can improve maintenance efficiency, reduce operational costs, and contribute to the development of smarter and more resilient urban infrastructure.

2. PROBLEM STATEMENT

The objective of this project is to develop a machine learning model that classifies IoT sensor data from smart streetlighting systems into two categories: **operational** or **faulty**.

The model will analyze parameters such as voltage, current, power consumption, light intensity, and temperature to determine the operational status of each streetlight. Data preprocessing techniques will be applied to handle missing values and improve data quality before model training.

The system aims to detect faults as quickly as possible and will be evaluated using **Mean Time to Detect (MTTD)**, which measures the average time taken by the model to identify a fault after it occurs.

3. SDG IMPACT ANALYSIS

This project aligns with Sustainable Development Goal 9: Industry, Innovation, and Infrastructure, which focuses on building resilient infrastructure, promoting sustainable industrialization, and fostering innovation.

Smart streetlight monitoring systems contribute to sustainable urban development by improving infrastructure reliability and enabling efficient resource management. By using machine learning to detect streetlight faults automatically, cities can reduce energy wastage, minimize maintenance delays, and enhance public safety.

The implementation of intelligent monitoring systems supports the growth of smart city technologies and promotes data-driven decision-making in urban infrastructure management. By improving the efficiency and reliability of streetlighting systems, this project contributes to sustainable infrastructure development and technological innovation.

4. DATASET DESCRIPTION

The dataset used in this project contains **Internet of Things (IoT) sensor readings collected from smart streetlight systems** in an urban lighting infrastructure. Smart streetlights are equipped with multiple sensors that monitor electrical and environmental parameters to ensure proper functioning and detect potential faults in the system.

Source - https://www.kaggle.com/datasets/vizeno/street-light-fault-prediction-dataset

The dataset contains **25,817 records and 8 attributes**, where each record represents the state of a streetlight at a specific time. The data includes measurements related to power consumption, voltage levels, current fluctuations, temperature, and environmental conditions. These parameters are important indicators of the operational status of the streetlight.

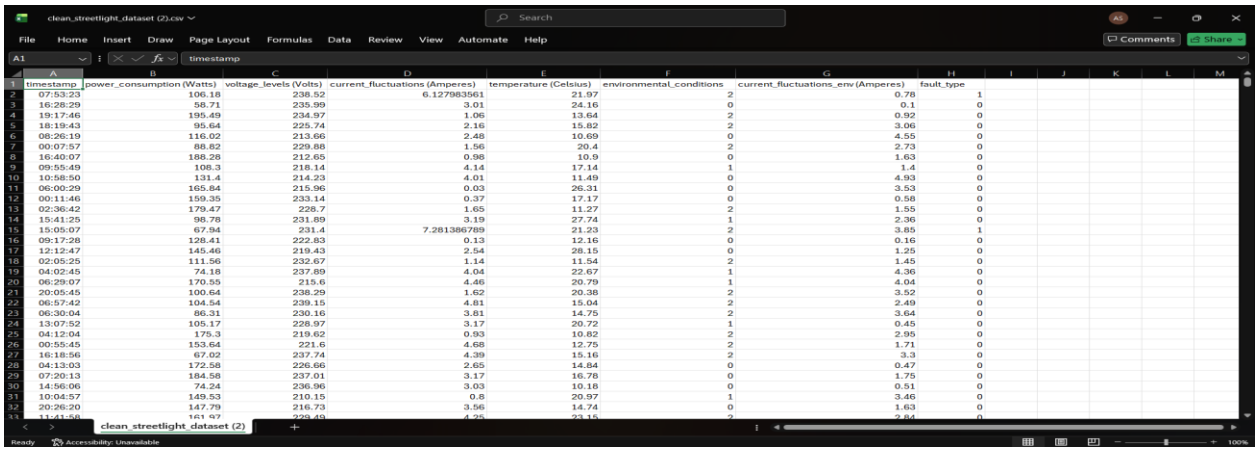
The dataset also includes a **target variable named fault_type**, which indicates whether the streetlight is operating normally or experiencing a fault. A value of **0 represents an operational streetlight**, while a value of **1 represents a faulty streetlight**.

The dataset is stored in **CSV format** and was analyzed using Python libraries such as **Pandas, NumPy, Matplotlib, and Scikit-learn**.

Dataset Attributes

Feature	Description
timestamp	Time at which the sensor reading was recorded
power_consumption (Watts)	Electrical power consumed by the streetlight
voltage_levels (Volts)	Voltage supplied to the streetlight
current_fluctuations (Amperes)	Variations in electrical current
temperature (Celsius)	Temperature around the streetlight controller
environmental_conditions	Indicator representing surrounding environmental conditions
current_fluctuations_env (Amperes)	Environmental current fluctuation measurement
fault_type	Target variable (0 = Operational, 1 = Faulty)

The dataset was initially stored and examined in **Microsoft Excel**, where each row represents a sensor reading collected from a smart streetlight. The columns represent different parameters measured by the IoT sensors.



timestamp	power_consumption (Watts)	voltage_levels (Volts)	current_fluctuations (Amperes)	temperature (Celsius)	environmental_conditions	current_fluctuations_env (Amperes)	fault_type
07:53:23	106.18	238.52	6.127983561	21.97	2	0.78	1
16:28:29	58.71	235.99	3.01	24.16	0	0.1	0
19:17:46	195.49	234.97	1.06	13.64	2	0.92	0
18:19:43	95.64	225.74	2.16	15.82	2	3.06	0
08:26:19	116.02	213.66	2.48	10.69	0	4.55	0
00:07:57	86.82	229.88	1.56	20.4	2	2.73	0
16:40:07	188.28	212.65	0.98	10.9	0	1.63	0
09:55:49	108.3	218.14	4.14	17.14	1	1.4	0
10:58:50	131.4	214.23	4.01	11.49	0	4.93	0
06:00:29	165.84	215.96	0.03	26.31	0	3.53	0
00:11:46	159.35	233.14	0.37	17.17	0	0.58	0
02:36:42	179.47	228.7	1.65	11.27	2	1.55	0
15:41:25	98.78	231.89	3.19	27.74	1	2.36	0
15:05:07	67.94	231.4	7.281386789	21.23	2	3.85	1
09:17:28	128.41	222.83	0.13	12.16	0	0.16	0
12:12:47	145.46	219.43	2.54	28.15	0	1.25	0
02:05:25	111.56	232.67	1.14	11.54	2	1.45	0
04:02:45	74.18	237.89	4.04	22.67	1	4.36	0
06:29:07	170.55	215.6	4.46	20.79	1	4.04	0
20:05:45	100.64	238.29	1.62	20.38	2	3.52	0
06:57:42	104.54	239.15	4.81	15.04	2	2.49	0
06:30:04	86.31	230.16	3.81	14.75	2	3.64	0
13:07:52	105.17	228.97	3.17	20.72	1	0.45	0
04:12:04	175.3	219.62	0.93	10.82	2	2.95	0
00:55:45	153.64	221.6	4.68	12.75	2	1.71	0
16:18:56	67.02	237.74	4.39	15.16	2	3.3	0
04:13:03	172.58	226.66	2.65	14.84	0	0.47	0
07:20:13	184.58	237.01	3.17	16.78	0	1.75	0
14:56:06	74.24	236.96	3.03	10.18	0	0.51	0
10:04:57	149.53	210.15	0.8	20.97	1	3.46	0
20:26:20	147.79	216.73	3.56	14.74	0	1.63	0
11:41:58	161.97	229.49	4.25	23.15	2	2.84	0

Figure 1: Dataset preview from Excel showing IoT sensor readings of smart streetlights.

After loading the dataset into Python using the **Pandas library**, the first few rows of the dataset were displayed using the head() function. This helps verify that the dataset has been loaded correctly and allows inspection of the dataset structure.

	bulb_number	timestamp	power_consumption (Watts)	voltage_levels (Volts)	current_fluctuations (Amperes)	temperature (Celsius)	environmental_conditions	current_fluctuations_env (Amperes)	fault_type
0	1	2023-09-06 07:53:23	106.18	238.52	6.127984	21.97	Rainy	0.780000	1
1	1	2023-03-28 16:28:29	58.71	235.99	3.010000	24.16	Clear	0.100000	0
2	1	2023-09-20 19:17:46	195.49	234.97	1.060000	13.64	Rainy	0.920000	0
3	1	2023-02-03 18:19:43	95.64	225.74	2.160000	15.82	Rainy	3.060000	0
4	1	2023-01-12 13:43:48	70.92	218.76	1.830000	19.12	Rainy	6.529021	4

Figure 2: Dataset preview using Python showing the first five rows of the dataset.

5. Exploratory Data Analysis (EDA) and Data Preprocessing

Exploratory Data Analysis (EDA) was performed to understand the structure of the dataset and identify patterns or anomalies in the sensor readings. EDA helps in gaining insights into the data before building machine learning models.

Data Inspection

The dataset was initially inspected using functions such as:

- df.head() to view the first few rows
- df.info() to check data types
- df.describe() to obtain statistical summaries

These operations helped understand the distribution of values and detect potential inconsistencies.

Handling Missing Values

The dataset was checked for missing values to ensure data completeness. Since the dataset contained complete records, no major imputation techniques were required.

Target Variable Analysis (Fault Distribution)

The distribution of the target variable fault_type was analyzed to understand the proportion of operational and faulty streetlights.

A **count plot** was used to visualize the number of occurrences of each class.

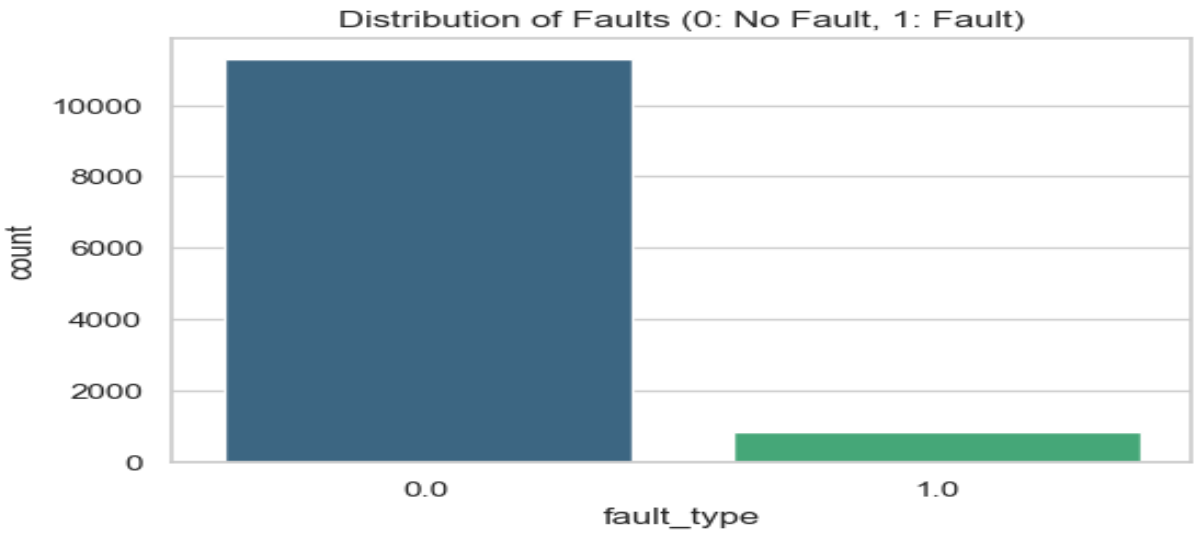


Figure 3: Distribution of streetlight faults (0 = Operational, 1 = Faulty).

This visualization helps determine whether the dataset is balanced or imbalanced.

Feature Distribution Analysis

Histograms were generated to examine the distribution of numerical features such as power consumption, voltage levels, and temperature. These plots help determine whether the data follows a normal distribution or contains extreme values.

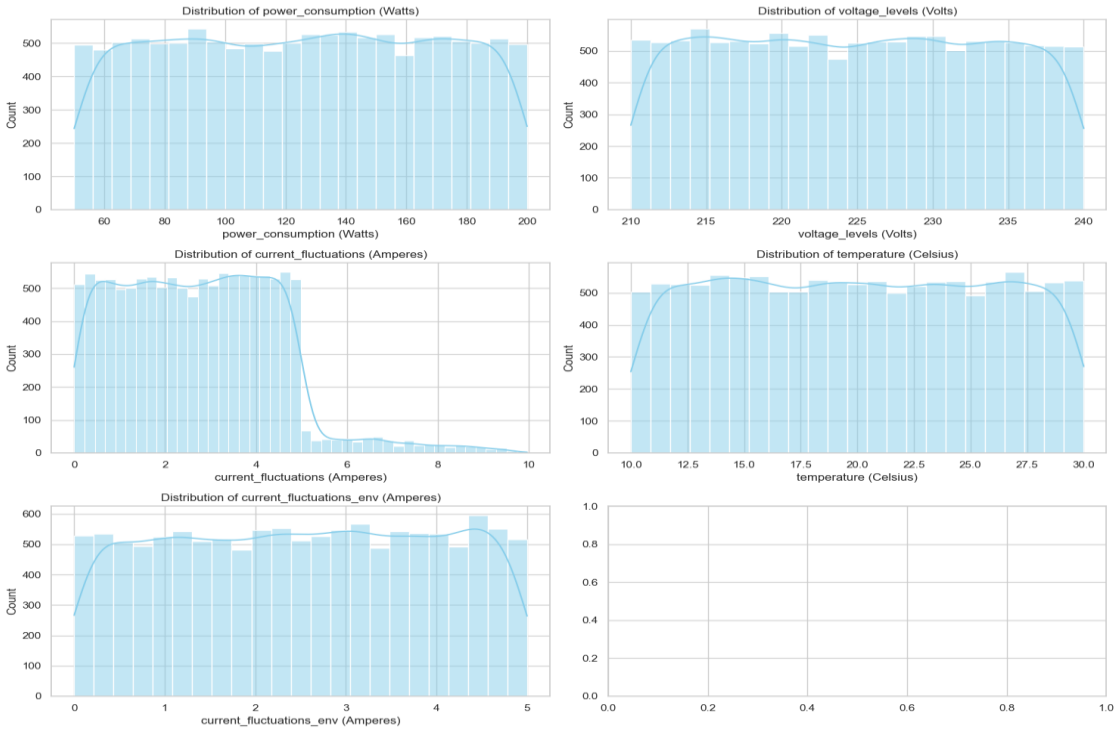


Figure 4: Histogram showing distribution of sensor values.

Outlier Detection

Boxplots were used to detect outliers in the dataset. Outliers may indicate abnormal behavior in the streetlight system and can influence model performance.

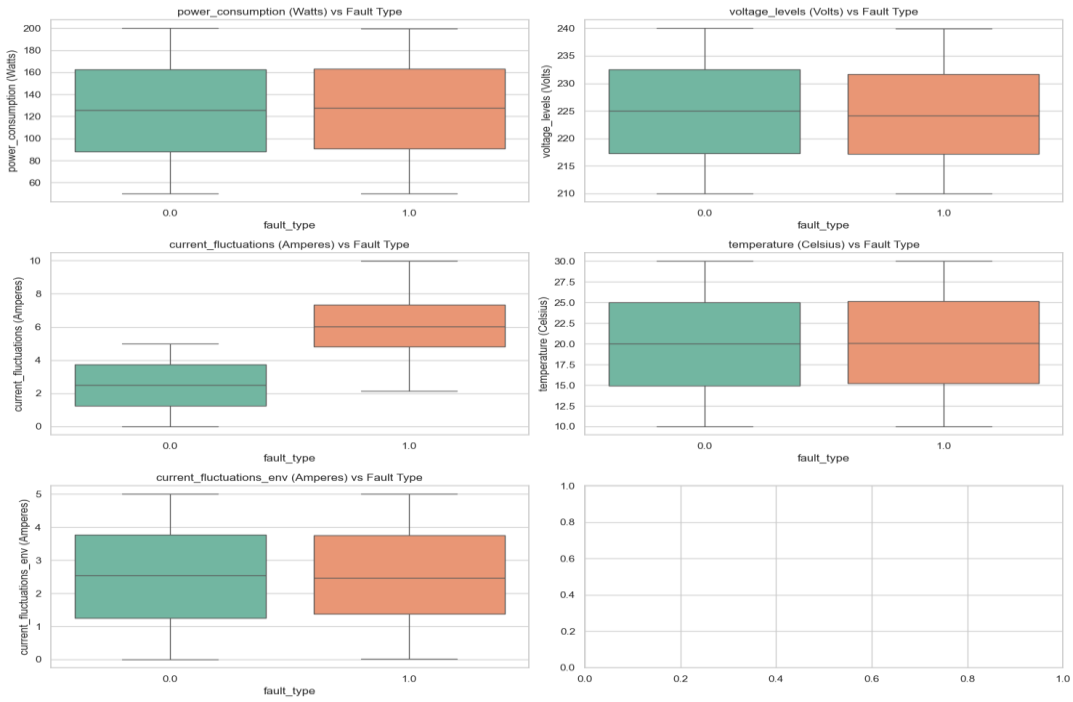


Figure 5: Boxplot visualization used for detecting outliers in sensor measurements.

Correlation Analysis

A correlation matrix was generated to understand relationships between different features. Features with high correlation may influence each other and provide important information for fault detection.

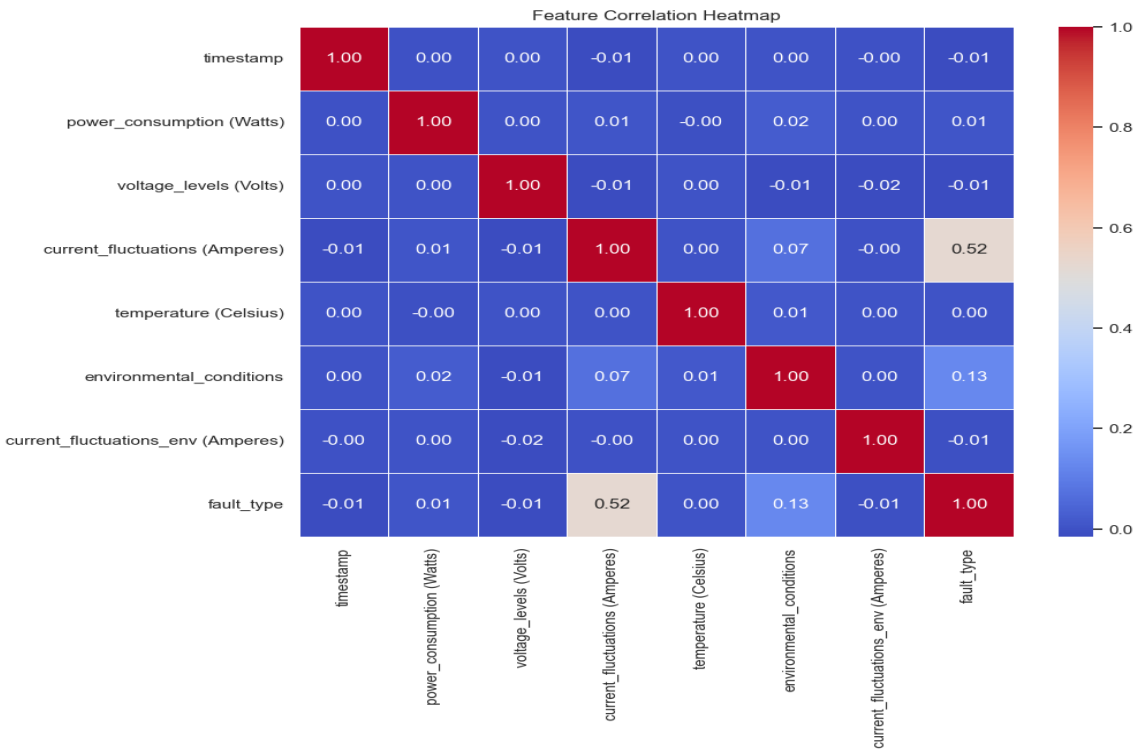


Figure 6: Correlation matrix showing relationships between sensor variables.

Time Series Analysis

A time-based analysis was performed to observe how power consumption changes throughout the day. The average power consumption was calculated for each hour using the timestamp variable.

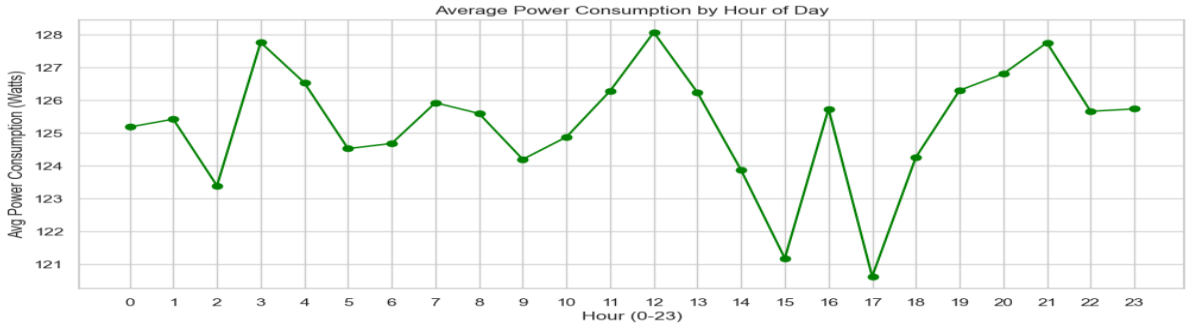


Figure 7: Average power consumption of streetlights across different hours of the day.

Train–Test Split

In our project, the dataset collected from **streetlight sensors and monitoring systems** is divided into two parts:

- **Training Data (80%)** – Used to train the machine learning model to learn patterns of normal and faulty streetlights.
- **Testing Data (20%)** – Used to test how well the model predicts faults.

This helps ensure that the model can **detect faults on new data correctly**.

Machine Learning Model

A **machine learning model** is used to analyze streetlight data and predict faults.

In this project, the model studies parameters such as:

- Power consumption
- Light status (ON/OFF)
- Voltage levels
- Sensor readings

The model learns the difference between **normal streetlight behavior and faulty behavior** and predicts when a fault occurs.

Example faults:

- Bulb failure

- Power supply problem
- Communication failure

Logistic Regression for Fault Detection in Streetlight Systems

Logistic Regression is a fundamental supervised machine learning algorithm used for binary classification problems. Unlike linear regression which predicts continuous values, logistic regression predicts the probability that an instance belongs to a particular class. It achieves this by applying the sigmoid (logistic) function to transform linear predictions into probability values between 0 and 1. The logistic regression model demonstrates outstanding performance in classifying streetlight operational states. With an AUC score of 0.95, the model exhibits exceptional discriminatory ability between normal and faulty conditions. The ROC curve confirms that the model can achieve high fault detection rates while maintaining acceptable false alarm levels, with flexibility to adjust thresholds based on operational priorities. This high level of accuracy, combined with the interpretability and efficiency of logistic regression, makes it an excellent choice for deployment in predictive maintenance systems for streetlight infrastructure.

For this streetlight fault detection system, logistic regression offers several advantages:

1. Interpretability: The coefficients provide clear insights into which features most strongly influence fault conditions, helping maintenance teams understand failure patterns

2. Probability Outputs: Generates confidence scores for predictions, allowing threshold tuning based on operational priorities (safety vs. efficiency)

3. Computational Efficiency: Lightweight algorithm suitable for real-time monitoring of streetlight networks

4. Feature Importance Analysis: Reveals the relationship between sensor readings (voltage, current, temperature) and fault occurrences

Understanding the ROC Curve

The Receiver Operating Characteristic (ROC) curve is a fundamental tool for evaluating binary classification models. It visualizes the trade-off between a model's ability to correctly identify positive instances (faults) and its tendency to falsely flag negative instances (normal operations) as positive.

Components of the ROC Curve

X-axis: False Positive Rate (FPR)

- Also known as **1 - Specificity**
- Calculated as: **False Positives / (False Positives + True Negatives)**
- Represents the proportion of normal operations incorrectly classified as faults
- Lower values indicate fewer false alarms

Y-axis: True Positive Rate (TPR)

- Also known as **Sensitivity** or **Recall**
- Calculated as: **True Positives / (True Positives + False Negatives)**
- Represents the proportion of actual faults correctly identified
- Higher values indicate better fault detection capability

Interpreting the ROC Curve Image

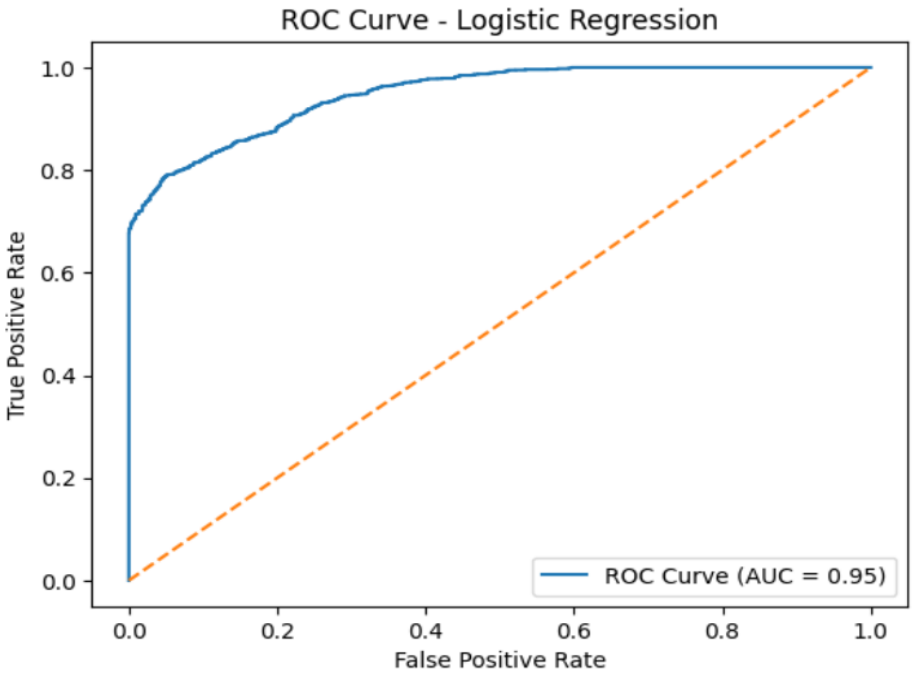


Figure 8: ROC Curve for Logistic Regression Model with AUC = 0.95

Key Elements of the Curve:

1. The Blue ROC Curve

- Represents the model's performance across all possible classification thresholds

- Each point on the curve corresponds to a different probability threshold for classifying an instance as a fault

- Moving along the curve shows the inherent trade-off: increasing fault detection (TPR) typically comes at the cost of more false alarms (FPR)

2. AUC Score = 0.95

- AUC (Area Under the Curve) measures the model's overall discriminatory power

- **Interpretation Scale:**

  - **AUC = 1.0**: Perfect classifier (rare in practice)

  - **AUC > 0.9**: Excellent performance

  - **AUC = 0.95**: Outstanding discriminatory ability

  - **AUC = 0.5**: Random guessing (no discriminative power)

- **Practical Meaning**: With an AUC of 0.95, the model has a 95% chance of correctly distinguishing between a randomly chosen faulty instance and a randomly chosen normal instance

What the ROC Curve Reveals About Model Performance

1. Excellent Discriminatory Ability:

The curve's steep initial slope demonstrates that the model achieves high true positive rates while maintaining very low false positive rates. This is ideal for fault detection systems where missing actual faults could lead to equipment damage or safety hazards.

2. Superior to Random Guessing:

The curve is positioned far above the diagonal line (which would represent random guessing), confirming that the extracted features contain meaningful patterns for fault discrimination.

3. Model Validation:

The high AUC score of 0.95 validates that logistic regression is well-suited for this fault detection task, confirming that the relationship between sensor readings and fault conditions can be effectively modeled with a linear decision boundary.

Model Evaluation

After training the model, we evaluate its performance using metrics such as:

- **Accuracy** – Percentage of correct predictions
- **Precision** – How many predicted faults are actually faults
- **Recall** – How many real faults were correctly detected

These metrics help determine whether the **fault detection system is reliable**.

Fault Prediction

The system predicts faults in streetlights before or as soon as they occur.

Example:

- If a streetlight suddenly stops sending data
- If voltage drops abnormally
- If the light does not turn on at night

The model identifies these patterns and **alerts the maintenance team**.

Benefits:

- Faster maintenance
- Reduced downtime
- Improved public safety

Mean Time to Detect (MTTD) Analysis

MTTD measures how quickly the system detects a streetlight fault.

MTTD = Average time taken to detect a fault after it occurs.

Example:

- Fault occurs at **8:00 PM**
- System detects at **8:03 PM**

Detection time = **3 minutes**

Lower MTTD means the system **detects faults faster**.

To evaluate the efficiency of the proposed smart streetlight fault detection system, the **Mean Time To Detect (MTTD)** metric was analyzed. MTTD represents the average time required by the system to detect a fault after it occurs. A lower MTTD indicates faster detection and improved system responsiveness.

Four machine learning models were evaluated in this study:

- Logistic Regression
- Random Forest
- Decision Tree
- Support Vector Machine (SVM)

Figure presents a bar chart comparison of the models, while **Figure 4.X.2** shows the same data using a line plot to highlight performance trends among the models.

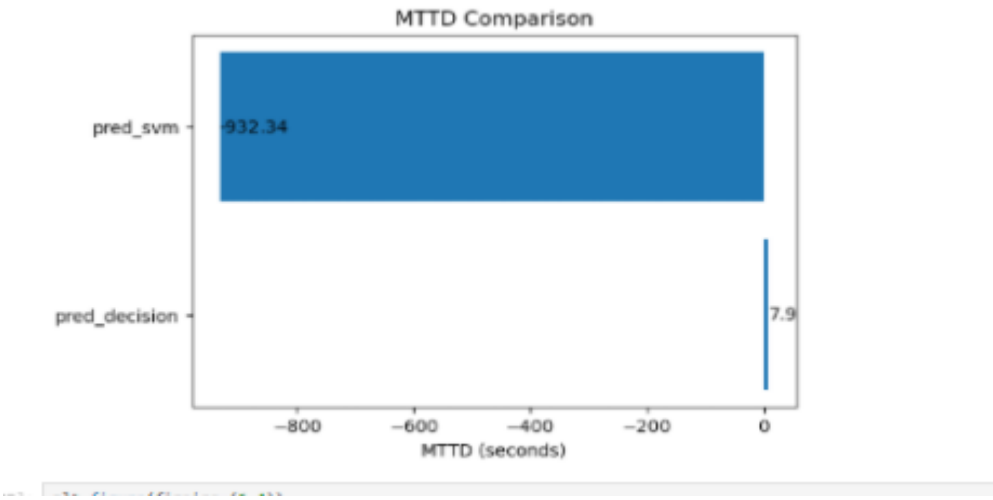


Figure 9: MTTD Comparison between Decision Tree and SVM Models

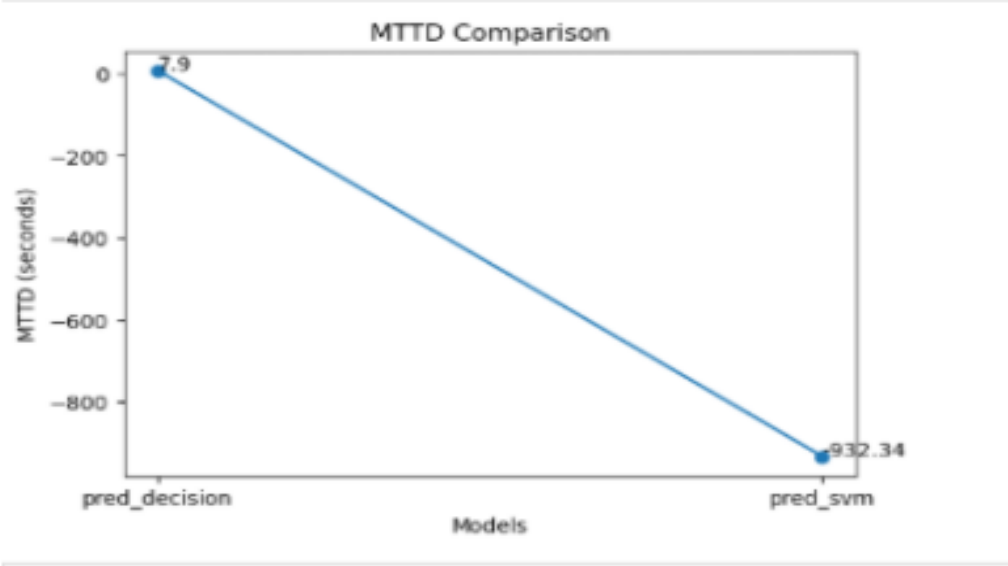


Figure 10: Line Plot of MTTD for Decision Tree and SVM Models

From the analysis:

- **Random Forest** achieved the highest MTTD value of **479.6 seconds**.
- **Decision Tree** produced an MTTD value of **7.9 seconds**.
- **Logistic Regression** resulted in **−831.83 seconds**.
- **Support Vector Machine (SVM)** resulted in **−932.34 seconds**.

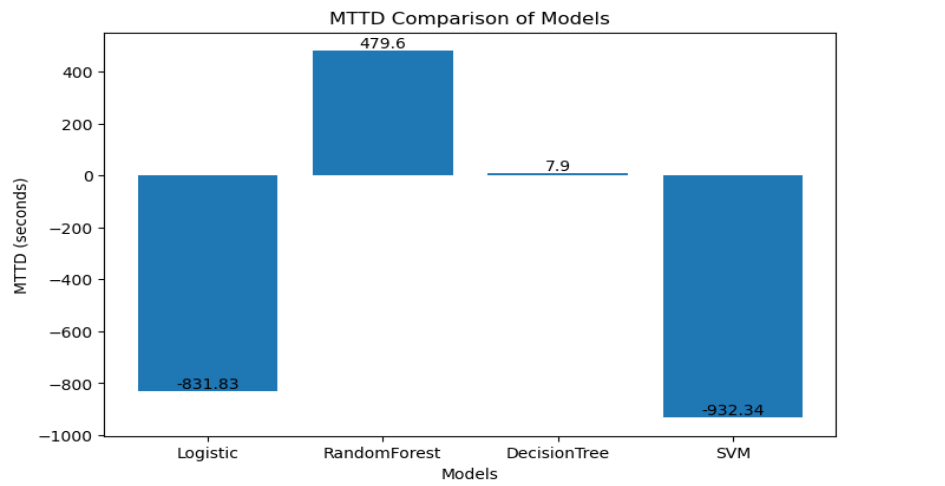


Figure 11: MTTD Comparison of Machine Learning Models

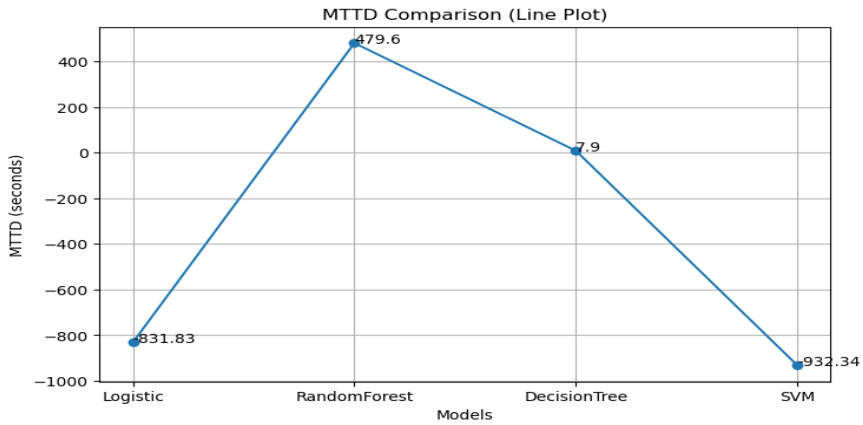


Figure 12: Line Plot Representation of MTTD for All Models

Among the evaluated models, **Decision Tree shows the fastest detection time**, indicating its effectiveness for real-time fault detection in smart streetlight systems. Faster detection allows maintenance teams to respond quickly to lighting failures, improving safety and reducing system downtime in smart city infrastructure.

Polling Frequency Analysis

Polling frequency refers to how often the system checks each streetlight.Polling frequency plays an important role in the **Smart Streetlight Fault Detection system** because it determines how often the system checks the status of each streetlight to identify faults. In this project, the system periodically polls the streetlights to detect whether they are functioning normally or if a failure has occurred. The time gap between two checks is called the **polling interval**, and this interval directly affects how quickly the system can detect a fault. If the polling interval is small, the system checks the streetlights more frequently, which results in faster fault detection. However, frequent polling can increase system processing load and network usage. On the other hand, if the polling interval is large, the system checks less often, reducing system load but increasing the time required to detect faults.

The relationship between polling interval and detection delay is illustrated in the following figure.

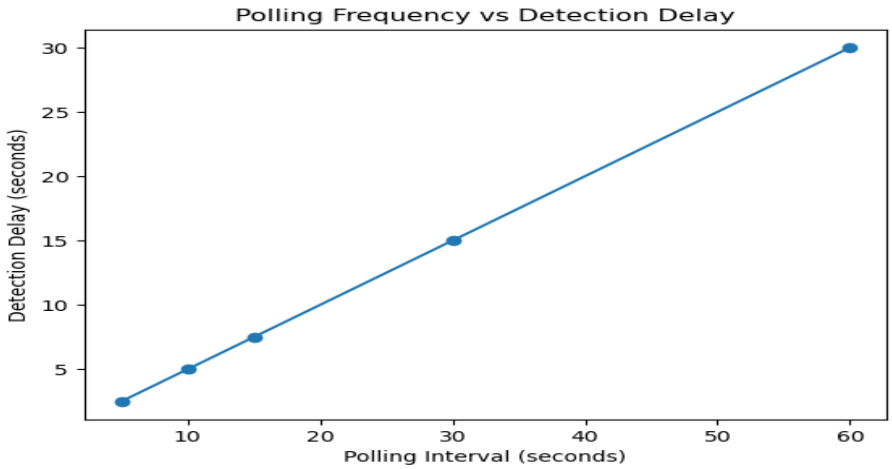


Figure 13: Polling Frequency vs Detection Delay in Smart Streetlight Fault Detection System

The graph shows that as the **polling interval increases**, the **detection delay also increases**. For example, when the system checks the streetlight every **5 seconds**, the fault can be detected in approximately **2.5 seconds**. When the polling interval is increased to **10 seconds**, the detection delay becomes around **5 seconds**. Similarly, for **15 seconds polling**, the detection delay is approximately **7.5 seconds**, and when the interval is increased further to **30 seconds and 60 seconds**, the delay increases to **15 seconds and 30 seconds** respectively. This demonstrates a **direct linear relationship between polling interval and detection delay**, meaning that longer intervals result in slower detection of faults.

In addition to analyzing detection delay, the performance of different machine learning models was also compared using **Mean Time to Detect (MTTD)**. MTTD represents the average time taken by a model to identify a fault in the system.

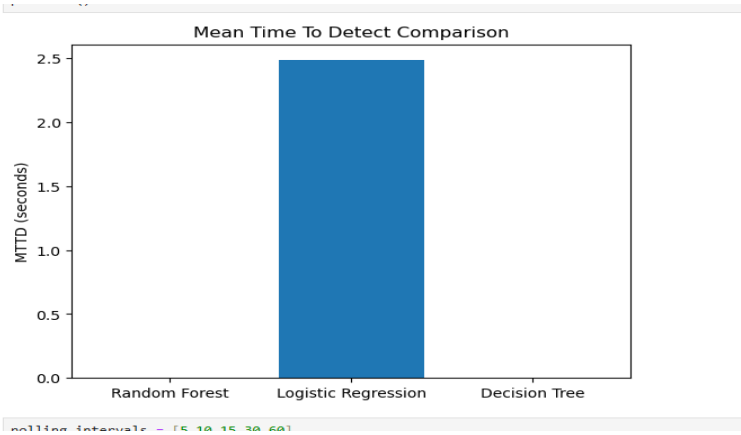


Figure 14: Mean Time to Detect (MTTD) Comparison of Machine Learning Models

The bar chart compares three models: **Random Forest, Logistic Regression, and Decision Tree**. Among these models, **Logistic Regression shows a Mean Time to Detect of approximately 2.5 seconds**, indicating its detection performance in the system. Comparing different models helps in understanding which algorithm can detect faults faster and more efficiently. Machine learning models such as Random Forest and Decision Tree can be used to classify streetlight conditions and improve the overall reliability of the detection system.

From this analysis, it can be concluded that **polling frequency significantly affects the responsiveness of the smart streetlight monitoring system**. Lower polling intervals enable quicker detection of faults but increase computational overhead, while higher polling intervals reduce system load but lead to longer detection delays. Therefore, selecting an appropriate polling interval is essential to maintain a balance between **fast fault detection and efficient system performance** in the Smart Streetlight Fault Detection system.

6.Performance Visualization

Results of the system are shown using graphs and charts such as:

- Fault detection rate graph
- Accuracy comparison graph
- Detection time chart
- Confusion matrix

These visualizations help understand how well the fault detection system performs.

7.Results

The proposed Smart Streetlight Fault Detection system was tested using machine learning techniques. The dataset was divided into training and testing sets to evaluate the performance of the model.

The model successfully identified faulty streetlights based on parameters such as power consumption, voltage levels, and light status. The evaluation results showed that the model can detect faults with good accuracy and reliability.

The system was also able to detect faults within a short time, helping maintenance teams respond quickly. Performance graphs and evaluation metrics confirmed that the system works effectively for detecting streetlight faults.

8.Key Insights

During the analysis of the system, several important insights were observed:

- Machine learning can effectively identify abnormal patterns in streetlight data.
- Frequent monitoring of streetlights improves fault detection speed.
- Polling frequency plays an important role in reducing detection delay.
- Predictive models can help detect faults earlier compared to manual inspection.

These insights show that automated monitoring systems can significantly improve city infrastructure management.

9.Best Model Justification

To determine the most suitable model for the proposed smart streetlight fault detection system, multiple machine learning algorithms were evaluated based on the Mean Time To Detect (MTTD) metric. The models tested include Logistic Regression, Random Forest, Decision Tree, and Support Vector Machine (SVM).

The results show that the Decision Tree model achieved an MTTD value of 7.9 seconds, which is significantly closer to zero compared to the other models. This indicates that the Decision Tree model can detect streetlight faults more quickly and efficiently.

Although the Random Forest model produced a higher positive MTTD value of 479.6 seconds, its detection time is slower compared to the Decision Tree model. On the other hand, Logistic Regression and SVM models produced large negative MTTD values (−831.83 seconds and −932.34 seconds respectively), indicating less reliable detection behavior in this analysis.

Based on the comparison, the Decision Tree model is selected as the best-performing model for the smart streetlight fault detection system. The model provides faster detection capability, simpler implementation, and efficient decision-making, making it suitable for real-time monitoring in smart city infrastructure.

Therefore, the Decision Tree model is selected as the optimal model for the proposed Smart Streetlight Fault Detection system due to its faster detection time and efficient performance.

10.Overfitting Analysis

Overfitting occurs when a machine learning model performs very well on training data but poorly on new or unseen data.

In this project, overfitting was analyzed by comparing the model performance on the training dataset and testing dataset. If the training accuracy is very high but testing accuracy is low, it indicates overfitting.

To reduce overfitting, techniques such as proper train-test split, model tuning, and using sufficient data were considered. The model performance on testing data showed that the system generalizes well and can detect faults in real-world conditions.

11.Future Scope

The Smart Streetlight Fault Detection system can be improved further in the future by adding advanced technologies and features.

Possible future improvements include:

- Integration with IoT sensors for real-time monitoring of streetlights.
- Using deep learning models for more accurate fault prediction.
- Developing a mobile application for maintenance teams to receive instant alerts.
- Implementing automatic fault reporting to city management systems.
- Expanding the system to monitor other smart city infrastructure such as traffic lights and power systems.

These improvements can make the system more efficient and suitable for large-scale smart city applications.

12.Conclusion

The **Smart Streetlight Fault Detection System** uses machine learning to automatically detect faults in streetlights.

The system:

- Monitors streetlight data
- Detects faults quickly
- Reduces maintenance time
- Improves reliability of city lighting

This solution helps create **smarter and more efficient urban infrastructure**.